Fittingly - AI-Powered Virtual Wardrobe & Fashion Recommendation Platform

Complete Product Documentation

Table of Contents

---

1. Executive Summary

Fittingly is a comprehensive AI-powered fashion e-commerce platform that combines virtual wardrobe management with intelligent outfit recommendations. The platform enables users to:

- Browse and purchase fashion products across multiple categories
- Manage a virtual 3D wardrobe with their clothing items
- Get AI-powered outfit combination suggestions
- Experience 60-minute express delivery
- Find nearby physical stores

Key Features

Feature	Description
Virtual Wardrobe	3D closet management with categorized clothing items
Smart Recommendations	AI-powered outfit combinations based on user's wardrobe
Product Catalog	Comprehensive fashion catalog with filtering and search
Shopping Cart	Full e-commerce cart functionality
Wishlist	Save favorite items for later
Order Management	Complete order lifecycle with tracking
Reviews & Ratings	Product review system with helpful votes

Feature	Description
Image Upload	Cloudinary-powered image management
Express Delivery	60-minute delivery option
Store Locator	Find nearby physical stores

---

2. System Architecture

High-Level Architecture Diagram

```
┌─────────────────────────────────────────────────────────────────────────────
────┐
│                              CLIENT LAYER
│
├─────────────────────────────────────────────────────────────────────────────
────┤
│  ┌───────────────────────────────────────────────────────────────────────┐
│  │                     Next.js 15 Frontend (React 19)
│ │
│  │  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐  │
│  │  │   Home   │ │ Wardrobe │ │  Search  │ │   Cart   │ │ Profile  │  │
│  │  └──────────┘ └──────────┘ └──────────┘ └──────────┘ └──────────┘  │
│  │  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐  │
│  │  │ Product  │ │ Checkout │ │ Wishlist │ │  Orders  │ │ Settings │  │
│  │  └──────────┘ └──────────┘ └──────────┘ └──────────┘ └──────────┘  │
│  └───────────────────────────────────────────────────────────────────────┘
│
└─────────────────────────────────────────────────────────────────────────────
────┘
                                        │
                                        │ HTTPS/REST API
                                        ▼
┌─────────────────────────────────────────────────────────────────────────────
────┐
│                                API LAYER
│
├─────────────────────────────────────────────────────────────────────────────
────┤
│  ┌───────────────────────────────────────────────────────────────────────┐
│  │                     Express.js Backend (Node.js)
│ │
│  │  ┌─────────────────────────────────────────────────────────────────┐    │
```

```
│
│  │  │                   Middleware Stack                    │
│   │
│  │  │  ┌────────┐ ┌────────┐ ┌────────┐ ┌────────┐ ┌────────────┐  │   │
│  │  │  │ Helmet │ │  CORS  │ │ Morgan │ │  Rate  │ │ Compression│  │   │
│  │  │  │Security│ │        │ │Logging │ │ Limit  │ │            │  │   │
│  │  │  └────────┘ └────────┘ └────────┘ └────────┘ └────────────┘  │   │
│  │  └─────────────────────────────────────────────────────────────┘   │
│  │  ┌─────────────────────────────────────────────────────────────┐   │
│  │  │                     API Routes                              │
│   │
│  │  │  ┌──────┐ ┌────────┐ ┌──────┐ ┌──────┐ ┌───────┐ ┌────────┐  │   │
│  │  │  │ Auth │ │Products│ │ Cart │ │Orders│ │Reviews│ │Wardrobe│  │   │
│  │  │  └──────┘ └────────┘ └──────┘ └──────┘ └───────┘ └────────┘  │   │
│  │  └─────────────────────────────────────────────────────────────┘   │
│  └───────────────────────────────────────────────────────────────────┘
│
└──────────────────────────────────────────────────────────────────────
────┘
                                │
                                │
        ┌───────────────────────┼───────────────────────┐
        │                       │                       │
        ▼                       ▼                       ▼
┌─────────────────────┐  ┌─────────────────────┐  ┌─────────────────────┐
│      MongoDB        │  │     Cloudinary      │  │    External APIs    │
│    (Database)       │  │  (Image Storage)    │  │  (Future: AI/ML)    │
│  ┌───────────────┐  │  │  ┌───────────────┐  │  │  ┌───────────────┐  │
│  │     Users     │  │  │  │   Products    │  │  │  │Recommendations│  │
│  │    Products   │  │  │  │   Wardrobe    │  │  │  │   Payments    │  │
│  │     Orders    │  │  │  │   Avatars     │  │  │  │   Shipping    │  │
│  │    Reviews    │  │  │  └───────────────┘  │  │  └───────────────┘  │
│  │    Wardrobe   │  │  └─────────────────────┘  └─────────────────────┘
│  └───────────────┘  │
└─────────────────────┘
```

Request Flow

```
User Action → Frontend Component → Context/State → API Client → Backend
Route
     ↓              ↓                    ↓              ↓            ↓
```

```
  UI Event    React Component    Context Provider   fetch()     Express
Router
     ↓              ↓                   ↓              ↓            ↓
  Handler     State Update       State Mutation    HTTP Req    Middleware
     ↓              ↓                   ↓              ↓            ↓
  Callback    Re-render          Persist/Sync      Response    Controller
                                                       ↓            ↓
                                                   JSON Data    MongoDB
```

---

3. Technology Stack

Backend Stack

Technology	Version	Purpose
Node.js	Latest	Runtime environment
Express.js	4.18.2	Web framework
TypeScript	5.3.3	Type safety
MongoDB	8.0.3 (Mongoose)	Database
JWT	9.0.2	Authentication
bcryptjs	2.4.3	Password hashing
Cloudinary	1.41.0	Image management
Helmet	7.1.0	Security headers
express-rate-limit	7.1.5	Rate limiting
express-validator	7.0.1	Input validation
Morgan	1.10.0	HTTP logging
Multer	1.4.5	File uploads

Frontend Stack

Technology	Version	Purpose
Next.js	15.5.4	React framework
React	19.1.0	UI library
TypeScript	5.x	Type safety
Tailwind CSS	4.x	Styling
Radix UI	Various	Accessible components
Lucide React	0.544.0	Icons

Technology	Version	Purpose
Sonner	2.0.7	Toast notifications
next-themes	0.4.6	Theme management

---

4. Backend Architecture

Directory Structure

```
backend/
├── src/
│   ├── config/
│   │   ├── database.ts      # MongoDB connection
│   │   └── cloudinary.ts    # Cloudinary configuration
│   ├── controllers/
│   │   ├── authController.ts
│   │   ├── productController.ts
│   │   ├── cartController.ts
│   │   ├── orderController.ts
│   │   ├── reviewController.ts
│   │   ├── wardrobeController.ts
│   │   └── uploadController.ts
│   ├── middleware/
│   │   ├── auth.ts          # JWT authentication
│   │   ├── errorHandler.ts  # Global error handling
│   │   └── validation.ts    # Request validation
│   ├── models/
│   │   ├── User.ts
│   │   ├── Product.ts
│   │   ├── Order.ts
│   │   ├── Review.ts
│   │   └── WardrobeItem.ts
│   ├── routes/
│   │   ├── auth.ts
│   │   ├── products.ts
│   │   ├── cart.ts
│   │   ├── orders.ts
│   │   ├── reviews.ts
│   │   ├── wardrobe.ts
│   │   └── upload.ts
│   ├── types/
│   │   └── index.ts         # TypeScript interfaces
│   ├── utils/
│   │   ├── jwt.ts           # Token utilities
│   │   ├── pagination.ts    # Pagination helpers
│   │   └── response.ts      # Response formatters
│   └── server.ts            # Application entry point
├── package.json
└── tsconfig.json
```

Server Configuration

```
// Key server configurations
const app = express();
const PORT = process.env.PORT || 5001;

// Security middleware
app.use(helmet());                    // Security headers
app.use(compression());               // Response compression

// CORS configuration
app.use(cors({
  origin: process.env.FRONTEND_URL || 'http://localhost:3000',
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'DELETE', 'PATCH', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization']
}));

// Rate limiting
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000,  // 15 minutes
  max: 100,                   // 100 requests per window
  message: { success: false, message: 'Too many requests' }
});
app.use('/api/', limiter);

// Body parsing
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ extended: true, limit: '10mb' }));
```

Environment Variables

```
# Server
PORT=5001
NODE_ENV=development

# Database
MONGODB_URI=mongodb://localhost:27017/fittingly

# JWT
JWT_SECRET=your-secret-key
JWT_EXPIRE=7d

# Cloudinary
CLOUDINARY_CLOUD_NAME=your-cloud-name
CLOUDINARY_API_KEY=your-api-key
CLOUDINARY_API_SECRET=your-api-secret

# Frontend
FRONTEND_URL=http://localhost:3000
```

```
# Rate Limiting
RATE_LIMIT_WINDOW_MS=900000
RATE_LIMIT_MAX_REQUESTS=100
```

---

5. Frontend Architecture

Directory Structure

```
frontend/
├── src/
│   ├── app/                    # Next.js App Router pages
│   │   ├── auth/
│   │   │   ├── login/
│   │   │   └── register/
│   │   ├── cart/
│   │   ├── checkout/
│   │   ├── delivery/
│   │   ├── help/
│   │   ├── notifications/
│   │   ├── order-confirmation/[id]/
│   │   ├── product/[id]/
│   │   ├── profile/
│   │   ├── search/
│   │   ├── settings/
│   │   ├── stores/
│   │   ├── wardrobe/
│   │   ├── wishlist/
│   │   ├── layout.tsx
│   │   ├── page.tsx
│   │   └── globals.css
│   ├── components/
│   │   ├── ui/                 # Radix UI components
│   │   │   ├── avatar.tsx
│   │   │   ├── badge.tsx
│   │   │   ├── button.tsx
│   │   │   ├── card.tsx
│   │   │   ├── dialog.tsx
│   │   │   ├── dropdown-menu.tsx
│   │   │   ├── input.tsx
│   │   │   ├── label.tsx
│   │   │   ├── navigation-menu.tsx
│   │   │   ├── select.tsx
│   │   │   ├── separator.tsx
│   │   │   ├── sheet.tsx
│   │   │   ├── sonner.tsx
│   │   │   └── tabs.tsx
│   │   ├── AccessibilityWrapper.tsx
│   │   ├── BottomNav.tsx
│   │   ├── Dummy3DModel.tsx
```

```
│   │   ├── ErrorBoundary.tsx
│   │   ├── Header.tsx
│   │   ├── LazyComponent.tsx
│   │   ├── LazyImage.tsx
│   │   ├── LoadingSpinner.tsx
│   │   └── RecommendationCard.tsx
│   ├── contexts/
│   │   ├── AuthContext.tsx
│   │   ├── CartContext.tsx
│   │   └── WishlistContext.tsx
│   ├── data/
│   │   └── products.ts         # Static product data
│   ├── hooks/
│   │   └── usePerformance.ts
│   ├── lib/
│   │   ├── api.ts              # API client
│   │   └── utils.ts            # Utility functions
│   └── utils/
├── public/
├── package.json
├── next.config.ts
├── tailwind.config.ts
└── tsconfig.json
```

Page Routes

Route	Page	Description
`/`	Home	Main product catalog with categories
`/auth/login`	Login	User authentication
`/auth/register`	Register	New user registration
`/product/[id]`	Product Detail	Individual product view with 3D model
`/search`	Search	Product search with filters
`/cart`	Cart	Shopping cart management
`/checkout`	Checkout	Order placement
`/order-confirmation/[id]`	Order Confirmation	Post-purchase confirmation
`/wardrobe`	Wardrobe	Virtual 3D closet management
`/wishlist`	Wishlist	Saved items
`/profile`	Profile	User profile and settings
`/settings`	Settings	App settings
`/notifications`	Notifications	User notifications
`/stores`	Stores	Nearby store locator

Route	Page	Description
`/delivery`	Delivery	60-min delivery info
`/help`	Help	Help and support

---

6. Data Models

User Model

```
interface IUser {
  _id: string;
  name: string;                    // Required, max 50 chars
  email: string;                   // Required, unique, validated
  password: string;                // Required, min 6 chars, hashed
  avatar?: string;                 // Profile image URL
  phone?: string;                  // Indian phone format
  dateOfBirth?: Date;
  gender?: 'male' | 'female' | 'other';
  preferences: {
    categories: ('men' | 'women' | 'kids' | 'unisex')[];
    sizes: string[];
    colors: string[];
    brands: string[];
  };
  wardrobe: ObjectId[];            // Ref: WardrobeItem
  wishlist: ObjectId[];            // Ref: Product
  cart: {
    items: {
      product: ObjectId;           // Ref: Product
      quantity: number;            // Min: 1
      size: string;
    }[];
    total: number;
  };
  orders: ObjectId[];              // Ref: Order
  isActive: boolean;               // Default: true
  createdAt: Date;
  updatedAt: Date;
}

// Indexes
- email: unique
- createdAt: descending
```

Product Model

```
interface IProduct {
  _id: string;
```

```
  name: string;                     // Required, max 100 chars
  description: string;              // Required, max 1000 chars
  price: number;                    // Required, min 0
  originalPrice?: number;           // For discount calculation
  discount?: number;                // 0-100%
  category: 'men' | 'women' | 'kids' | 'unisex';
  subcategory: 'topwear' | 'bottomwear' | 'footwear' | 'accessories';
  brand: string;                    // Required
  images: string[];                 // Min 1 required
  colors: string[];                 // Min 1 required
  sizes: string[];                  // Min 1 required
  rating: number;                   // 0-5, auto-calculated
  reviewCount: number;              // Auto-calculated
  features: string[];               // Product features
  fitTags: string[];                // Fit recommendations
  combinations: {                   // Outfit combinations
    id: string;
    color: string;
    bottom: string;
  }[];
  outfits: number;                  // Number of possible outfits
  delivery: string;                 // Delivery info text
  isActive: boolean;                // Default: true
  createdAt: Date;
  updatedAt: Date;
}

// Indexes
- Text search: name, description, brand
- Compound: category + subcategory
- Single: price, rating, createdAt, brand
```

Order Model

```
interface IOrder {
  _id: string;
  user: ObjectId;                   // Ref: User, required
  items: {
    product: ObjectId;              // Ref: Product
    quantity: number;               // Min: 1
    size: string;
    price: number;                  // Snapshot at order time
  }[];
  totalAmount: number;              // Required, min 0
  discount: number;                 // Default: 0
  shippingAddress: {
    name: string;                   // Required, 2-50 chars
    address: string;                // Required, 10-200 chars
    city: string;                   // Required, 2-50 chars
    state: string;                  // Required, 2-50 chars
    pincode: string;                // Required, 6 digits
```

```
    phone: string;                  // Required, Indian format
  };
  paymentMethod: 'credit_card' | 'debit_card' | 'upi' | 'net_banking' |
'wallet' | 'cod';
  paymentStatus: 'pending' | 'completed' | 'failed' | 'refunded';
  orderStatus: 'pending' | 'confirmed' | 'shipped' | 'delivered' |
'cancelled';
  trackingNumber?: string;
  createdAt: Date;
  updatedAt: Date;
}

// Virtual fields
- orderNumber: ORD-{last 8 chars of _id}
- estimatedDelivery: 3 days after shipping

// Indexes
- Compound: user + createdAt (descending)
- Single: orderStatus, paymentStatus, trackingNumber
```

Review Model

```
interface IReview {
  _id: string;
  user: ObjectId;                 // Ref: User, required
  product: ObjectId;              // Ref: Product, required
  rating: number;                 // Required, 1-5
  comment: string;                // Required, 10-500 chars
  images?: string[];              // Review images
  isVerified: boolean;            // Verified purchase
  helpful: number;                // Helpful votes count
  createdAt: Date;
  updatedAt: Date;
}

// Constraints
- Unique: user + product (one review per user per product)

// Virtual fields
- timeAgo: Human-readable time since review

// Indexes
- Unique compound: user + product
- Compound: product + createdAt (descending)
- Single: rating, helpful
```

WardrobeItem Model

```
interface IWardrobeItem {
  _id: string;
  user: ObjectId;                    // Ref: User, required
  name: string;                      // Required, max 100 chars
  category: 'upper' | 'lower' | 'shoes';
  subtype: string;                   // e.g., 'Tshirt', 'Jeans', 'Sneakers'
  color: string;                     // Required
  image: string;                     // Required, Cloudinary URL
  brand?: string;
  size?: string;
  purchaseDate?: Date;
  price?: number;                    // Min: 0
  tags: string[];                    // Custom tags
  isActive: boolean;                 // Default: true
  createdAt: Date;
  updatedAt: Date;
}

// Virtual fields
- ageInDays: Days since purchase

// Indexes
- Compound: user + category
- Compound: user + subtype
- Compound: user + createdAt (descending)
```

---

7. API Reference

Base URL

```
Development: http://localhost:5001/api
Production: https://api.fittingly.com/api
```

Response Format

All API responses follow this structure:

```
interface ApiResponse<T> {
  success: boolean;
  message: string;
  data?: T;
  error?: string;
  pagination?: {
    page: number;
    limit: number;
    total: number;
    pages: number;
```

```
    hasNext: boolean;
    hasPrev: boolean;
  };
}
```

Authentication API

Register User

```
POST /api/auth/register
Content-Type: application/json

Request Body:
{
  "name": "John Doe",              // Required, 2-50 chars
  "email": "john@example.com",  // Required, valid email
  "password": "password123",     // Required, min 6 chars
  "phone": "9876543210",         // Optional, Indian format
  "gender": "male",              // Optional: male|female|other
  "dateOfBirth": "1990-01-15"    // Optional, ISO date
}

Response (201):
{
  "success": true,
  "message": "User registered successfully",
  "data": {
    "user": { ... },
    "token": "eyJhbGciOiJIUzI1NiIs..."
  }
}
```

Login

```
POST /api/auth/login
Content-Type: application/json

Request Body:
{
  "email": "john@example.com",
  "password": "password123"
}

Response (200):
{
  "success": true,
  "message": "Login successful",
  "data": {
```

```
    "user": { ... },
    "token": "eyJhbGciOiJIUzI1NiIs..."
  }
}
```

Get Current User

```
GET /api/auth/me
Authorization: Bearer <token>

Response (200):
{
  "success": true,
  "message": "User profile retrieved successfully",
  "data": {
    "_id": "...",
    "name": "John Doe",
    "email": "john@example.com",
    "preferences": { ... },
    "wardrobe": [...],
    "wishlist": [...],
    "cart": { ... },
    "orders": [...]
  }
}
```

Update Profile

```
PUT /api/auth/profile
Authorization: Bearer <token>
Content-Type: application/json

Request Body:
{
  "name": "John Updated",
  "phone": "9876543211",
  "preferences": {
    "categories": ["men"],
    "sizes": ["M", "L"],
    "colors": ["blue", "black"],
    "brands": ["Nike", "Adidas"]
  }
}

Response (200):
{
  "success": true,
  "message": "Profile updated successfully",
```

```
  "data": { ... }
}
```

Change Password

```
PUT /api/auth/change-password
Authorization: Bearer <token>
Content-Type: application/json

Request Body:
{
  "currentPassword": "oldpassword",
  "newPassword": "newpassword123"
}

Response (200):
{
  "success": true,
  "message": "Password changed successfully"
}
```

Logout

```
POST /api/auth/logout
Authorization: Bearer <token>

Response (200):
{
  "success": true,
  "message": "Logout successful"
}
```

Products API

Get All Products

```
GET /api/products
Query Parameters:
  - category: men|women|kids|unisex
  - subcategory: topwear|bottomwear|footwear|accessories
  - brand: string (partial match)
  - minPrice: number
  - maxPrice: number
  - colors: comma-separated string
  - sizes: comma-separated string
  - rating: number (minimum rating)
```

```
  - search: string (text search)
  - sortBy: price|rating|newest|popular
  - sortOrder: asc|desc
  - page: number (default: 1)
  - limit: number (default: 10, max: 100)

Response (200):
{
  "success": true,
  "message": "Products retrieved successfully",
  "data": [...],
  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 100,
    "pages": 10,
    "hasNext": true,
    "hasPrev": false
  }
}
```

Get Single Product

```
GET /api/products/:id

Response (200):
{
  "success": true,
  "message": "Product retrieved successfully",
  "data": {
    "_id": "...",
    "name": "Puma Sunset Road T-Shirt",
    "description": "...",
    "price": 874,
    "originalPrice": 1999,
    "discount": 56,
    "category": "men",
    "subcategory": "topwear",
    "brand": "Puma",
    "images": ["..."],
    "colors": ["blue", "black"],
    "sizes": ["S", "M", "L", "XL"],
    "rating": 4.5,
    "reviewCount": 120,
    "features": ["Your biceps look bigger", "Fabric Cotton"],
    "fitTags": ["Best Fit", "Good Price"],
    "combinations": [...],
    "outfits": 14,
    "delivery": "Deliver by 10 Aug 2025"
  }
}
```

Get Categories

```
GET /api/products/categories

Response (200):
{
  "success": true,
  "message": "Categories retrieved successfully",
  "data": {
    "categories": ["men", "women", "kids", "unisex"],
    "subcategories": ["topwear", "bottomwear", "footwear", "accessories"],
    "brands": ["Puma", "Nike", "Adidas", ...]
  }
}
```

Get Featured Products

```
GET /api/products/featured?limit=8

Response (200):
{
  "success": true,
  "message": "Featured products retrieved successfully",
  "data": [...]
}
```

Get Related Products

```
GET /api/products/:id/related

Response (200):
{
  "success": true,
  "message": "Related products retrieved successfully",
  "data": [...]  // Max 4 products
}
```

Create Product (Admin)

```
POST /api/products
Authorization: Bearer <admin_token>
Content-Type: application/json
```

```
Request Body:
{
  "name": "New Product",
  "description": "Product description...",
  "price": 999,
  "originalPrice": 1499,
  "category": "men",
  "subcategory": "topwear",
  "brand": "BrandName",
  "images": ["url1", "url2"],
  "colors": ["red", "blue"],
  "sizes": ["S", "M", "L"]
}

Response (201):
{
  "success": true,
  "message": "Product created successfully",
  "data": { ... }
}
```

Update Product (Admin)

```
PUT /api/products/:id
Authorization: Bearer <admin_token>
Content-Type: application/json

Request Body: { ... partial product data ... }

Response (200):
{
  "success": true,
  "message": "Product updated successfully",
  "data": { ... }
}
```

Delete Product (Admin)

```
DELETE /api/products/:id
Authorization: Bearer <admin_token>

Response (200):
{
  "success": true,
  "message": "Product deleted successfully"
}
```

Cart API

Get Cart

```
GET /api/cart
Authorization: Bearer <token>

Response (200):
{
  "success": true,
  "message": "Cart retrieved successfully",
  "data": {
    "items": [
      {
        "product": {
          "_id": "...",
          "name": "Product Name",
          "price": 999,
          "images": ["..."],
          "brand": "Brand"
        },
        "quantity": 2,
        "size": "M"
      }
    ],
    "total": 1998
  }
}
```

Add to Cart

```
POST /api/cart
Authorization: Bearer <token>
Content-Type: application/json

Request Body:
{
  "productId": "product_id_here",
  "quantity": 1,
  "size": "M"
}

Response (200):
{
  "success": true,
  "message": "Item added to cart successfully",
  "data": {
    "items": [...],
    "total": 999
```

```
  }
}
```

Update Cart Item

```
PUT /api/cart/:itemId
Authorization: Bearer <token>
Content-Type: application/json

Request Body:
{
  "quantity": 3
}

Response (200):
{
  "success": true,
  "message": "Cart item updated successfully",
  "data": {
    "items": [...],
    "total": 2997
  }
}
```

Remove from Cart

```
DELETE /api/cart/:itemId
Authorization: Bearer <token>

Response (200):
{
  "success": true,
  "message": "Item removed from cart successfully",
  "data": {
    "items": [...],
    "total": 0
  }
}
```

Clear Cart

```
DELETE /api/cart
Authorization: Bearer <token>

Response (200):
{
```

```
  "success": true,
  "message": "Cart cleared successfully",
  "data": {
    "items": [],
    "total": 0
  }
}
```

Get Cart Count

```
GET /api/cart/count
Authorization: Bearer <token>

Response (200):
{
  "success": true,
  "message": "Cart count retrieved successfully",
  "data": {
    "count": 5
  }
}
```

Orders API

Create Order

```
POST /api/orders
Authorization: Bearer <token>
Content-Type: application/json

Request Body:
{
  "shippingAddress": {
    "name": "John Doe",
    "address": "123 Main Street, Apartment 4B",
    "city": "Mumbai",
    "state": "Maharashtra",
    "pincode": "400001",
    "phone": "9876543210"
  },
  "paymentMethod": "upi"  //
credit_card|debit_card|upi|net_banking|wallet|cod
}

Response (201):
{
  "success": true,
  "message": "Order created successfully",
```

```
  "data": {
    "_id": "...",
    "orderNumber": "ORD-ABC12345",
    "items": [...],
    "totalAmount": 2997,
    "orderStatus": "pending",
    "paymentStatus": "pending",
    "createdAt": "2025-01-15T10:30:00Z"
  }
}
```

Get User Orders

```
GET /api/orders
Authorization: Bearer <token>
Query Parameters:
  - status: pending|confirmed|shipped|delivered|cancelled
  - page: number
  - limit: number

Response (200):
{
  "success": true,
  "message": "Orders retrieved successfully",
  "data": [...],
  "pagination": { ... }
}
```

Get Single Order

```
GET /api/orders/:id
Authorization: Bearer <token>

Response (200):
{
  "success": true,
  "message": "Order retrieved successfully",
  "data": {
    "_id": "...",
    "orderNumber": "ORD-ABC12345",
    "items": [
      {
        "product": { ... },
        "quantity": 2,
        "size": "M",
        "price": 999
      }
    ],
```

```
    "totalAmount": 1998,
    "shippingAddress": { ... },
    "paymentMethod": "upi",
    "paymentStatus": "completed",
    "orderStatus": "shipped",
    "trackingNumber": "TRK123456",
    "estimatedDelivery": "2025-01-18T00:00:00Z"
  }
}
```

Update Order Status

```
PUT /api/orders/:id/status
Authorization: Bearer <token>
Content-Type: application/json

Request Body:
{
  "status": "shipped",
  "trackingNumber": "TRK123456"  // Optional
}

Response (200):
{
  "success": true,
  "message": "Order status updated successfully",
  "data": { ... }
}
```

Cancel Order

```
PUT /api/orders/:id/cancel
Authorization: Bearer <token>

Response (200):
{
  "success": true,
  "message": "Order cancelled successfully",
  "data": {
    "orderStatus": "cancelled",
    "paymentStatus": "refunded"
  }
}

Error (400) - If already shipped/delivered:
{
  "success": false,
```

```
  "message": "Cannot cancel order that has been shipped or delivered"
}
```

Get Order Statistics

```
GET /api/orders/stats
Authorization: Bearer <token>

Response (200):
{
  "success": true,
  "message": "Order statistics retrieved successfully",
  "data": {
    "totalOrders": 24,
    "totalSpent": 45000,
    "statusBreakdown": [
      { "_id": "delivered", "count": 20, "totalAmount": 40000 },
      { "_id": "pending", "count": 2, "totalAmount": 3000 },
      { "_id": "cancelled", "count": 2, "totalAmount": 2000 }
    ]
  }
}
```

Reviews API

Get Product Reviews

```
GET /api/reviews/product/:productId
Query Parameters:
  - rating: number (filter by rating)
  - page: number
  - limit: number

Response (200):
{
  "success": true,
  "message": "Product reviews retrieved successfully",
  "data": {
    "reviews": [
      {
        "_id": "...",
        "user": {
          "_id": "...",
          "name": "John Doe",
          "avatar": "..."
        },
        "rating": 5,
        "comment": "Great product!",
```

```
        "images": ["..."],
        "helpful": 12,
        "timeAgo": "2 days ago",
        "createdAt": "2025-01-13T10:30:00Z"
      }
    ],
    "ratingDistribution": {
      "5": 45,
      "4": 30,
      "3": 15,
      "2": 7,
      "1": 3
    },
    "averageRating": 4.2,
    "totalReviews": 100
  },
  "pagination": { ... }
}
```

Create Review

```
POST /api/reviews
Authorization: Bearer <token>
Content-Type: application/json

Request Body:
{
  "productId": "product_id_here",
  "rating": 5,
  "comment": "Excellent quality! Fits perfectly.",
  "images": ["url1", "url2"]  // Optional
}

Response (201):
{
  "success": true,
  "message": "Review created successfully",
  "data": { ... }
}

Error (400) - If already reviewed:
{
  "success": false,
  "message": "You have already reviewed this product"
}
```

Update Review

```
PUT /api/reviews/:id
Authorization: Bearer <token>
Content-Type: application/json

Request Body:
{
  "rating": 4,
  "comment": "Updated review comment",
  "images": ["url1"]
}

Response (200):
{
  "success": true,
  "message": "Review updated successfully",
  "data": { ... }
}
```

Delete Review

```
DELETE /api/reviews/:id
Authorization: Bearer <token>

Response (200):
{
  "success": true,
  "message": "Review deleted successfully"
}
```

Get My Reviews

```
GET /api/reviews/my
Authorization: Bearer <token>
Query Parameters:
  - page: number
  - limit: number

Response (200):
{
  "success": true,
  "message": "User reviews retrieved successfully",
  "data": [...],
  "pagination": { ... }
}
```

Mark Review Helpful

```
POST /api/reviews/:id/helpful
Authorization: Bearer <token>

Response (200):
{
  "success": true,
  "message": "Review marked as helpful",
  "data": {
    "helpful": 13
  }
}
```

Get Review Statistics

```
GET /api/reviews/stats
Authorization: Bearer <token>

Response (200):
{
  "success": true,
  "message": "Review statistics retrieved successfully",
  "data": {
    "totalReviews": 15,
    "averageRating": 4.3,
    "ratingBreakdown": {
      "5": 8,
      "4": 4,
      "3": 2,
      "2": 1,
      "1": 0
    }
  }
}
```

Wardrobe API

Get Wardrobe Items

```
GET /api/wardrobe
Authorization: Bearer <token>
Query Parameters:
  - category: upper|lower|shoes
  - subtype: string
  - page: number
  - limit: number

Response (200):
```

```
{
  "success": true,
  "message": "Wardrobe items retrieved successfully",
  "data": [
    {
      "_id": "...",
      "name": "Black T-Shirt",
      "category": "upper",
      "subtype": "Tshirt",
      "color": "#111827",
      "image": "cloudinary_url",
      "brand": "Nike",
      "size": "M",
      "tags": ["casual", "summer"],
      "ageInDays": 45
    }
  ],
  "pagination": { ... }
}
```

Get Single Wardrobe Item

```
GET /api/wardrobe/:id
Authorization: Bearer <token>

Response (200):
{
  "success": true,
  "message": "Wardrobe item retrieved successfully",
  "data": { ... }
}
```

Create Wardrobe Item

```
POST /api/wardrobe
Authorization: Bearer <token>
Content-Type: application/json

Request Body:
{
  "name": "Blue Denim Jeans",
  "category": "lower",
  "subtype": "Jeans",
  "color": "#3b82f6",
  "image": "cloudinary_url",
  "brand": "Levi's",
  "size": "32",
  "purchaseDate": "2024-12-01",
```

```
    "price": 2499,
    "tags": ["casual", "denim"]
  }

  Response (201):
  {
    "success": true,
    "message": "Wardrobe item created successfully",
    "data": { ... }
  }
```

Update Wardrobe Item

```
PUT /api/wardrobe/:id
Authorization: Bearer <token>
Content-Type: application/json

Request Body: { ... partial wardrobe item data ... }

Response (200):
{
  "success": true,
  "message": "Wardrobe item updated successfully",
  "data": { ... }
}
```

Delete Wardrobe Item

```
DELETE /api/wardrobe/:id
Authorization: Bearer <token>

Response (200):
{
  "success": true,
  "message": "Wardrobe item deleted successfully"
}
```

Get Wardrobe Statistics

```
GET /api/wardrobe/stats
Authorization: Bearer <token>

Response (200):
{
  "success": true,
  "message": "Wardrobe statistics retrieved successfully",
```

```
  "data": {
    "totalItems": 45,
    "categoryStats": {
      "upper": 20,
      "lower": 15,
      "shoes": 10
    },
    "breakdown": [
      { "_id": "upper", "count": 20, "items": [...] },
      { "_id": "lower", "count": 15, "items": [...] },
      { "_id": "shoes", "count": 10, "items": [...] }
    ]
  }
}
```

Get Wardrobe by Category

```
GET /api/wardrobe/category/:category
Authorization: Bearer <token>
Query Parameters:
  - subtype: string (optional)

Response (200):
{
  "success": true,
  "message": "Wardrobe items retrieved successfully",
  "data": [...]
}
```

Upload API

Upload Single Image

```
POST /api/upload/image
Authorization: Bearer <token>
Content-Type: multipart/form-data

Form Data:
  - image: File (required)
  - folder: string (optional, default: "fittingly")

Response (200):
{
  "success": true,
  "message": "Image uploaded successfully",
  "data": {
    "url": "https://res.cloudinary.com/...",
    "originalName": "photo.jpg",
```

```
    "size": 245678,
    "mimetype": "image/jpeg"
  }
}
```

Upload Multiple Images

```
POST /api/upload/images
Authorization: Bearer <token>
Content-Type: multipart/form-data

Form Data:
  - images: File[] (max 10 files)
  - folder: string (optional)

Response (200):
{
  "success": true,
  "message": "Images uploaded successfully",
  "data": {
    "images": [
      { "url": "...", "originalName": "...", "size": ..., "mimetype":
"..." }
    ],
    "count": 3
  }
}
```

Upload Wardrobe Image

```
POST /api/upload/wardrobe
Authorization: Bearer <token>
Content-Type: multipart/form-data

Form Data:
  - image: File (required)

Response (200):
{
  "success": true,
  "message": "Wardrobe image uploaded successfully",
  "data": {
    "url": "https://res.cloudinary.com/fittingly/wardrobe/...",
    ...
  }
}
```

Upload Product Image

```
POST /api/upload/product
Authorization: Bearer <token>
Content-Type: multipart/form-data

Form Data:
  - image: File (required)

Response (200):
{
  "success": true,
  "message": "Product image uploaded successfully",
  "data": {
    "url": "https://res.cloudinary.com/fittingly/products/...",
    ...
  }
}
```

Delete Image

```
DELETE /api/upload/image
Authorization: Bearer <token>
Content-Type: application/json

Request Body:
{
  "publicId": "fittingly/wardrobe/image_id"
}

Response (200):
{
  "success": true,
  "message": "Image deleted successfully"
}
```

Get Optimized Image URL

```
GET /api/upload/optimize
Query Parameters:
  - publicId: string (required)
  - width: number (default: 400)
  - height: number (default: 400)
  - quality: number (default: 80)
  - format: string (default: "auto")

Response (200):
```

```json
{
  "success": true,
  "message": "Optimized image URL generated",
  "data": {
    "url": "https://res.cloudinary.com/...",
    "publicId": "...",
    "options": {
      "width": 400,
      "height": 400,
      "quality": 80,
      "format": "auto"
    }
  }
}
```

---

8. Authentication & Security

JWT Authentication Flow

```
1. User Login/Register
   └─> Server validates credentials
       └─> Server generates JWT token (7 days expiry)
           └─> Token returned to client
               └─> Client stores in localStorage

2. Authenticated Request
   └─> Client includes token in Authorization header
       └─> Server middleware validates token
           └─> Server extracts user from token
               └─> Request proceeds to controller
```

Token Structure

```javascript
// JWT Payload
{
  userId: string,    // MongoDB ObjectId
  iat: number,       // Issued at timestamp
  exp: number        // Expiration timestamp (7 days)
}

// Token Generation
const token = jwt.sign(
  { userId },
  process.env.JWT_SECRET,
  { expiresIn: '7d' }
);
```

Security Middleware

```javascript
// Authentication Middleware
export const authenticate = async (req, res, next) => {
  const token = req.header('Authorization')?.replace('Bearer ', '');

  if (!token) {
    return res.status(401).json({
      success: false,
      message: 'Access denied. No token provided.'
    });
  }

  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  const user = await User.findById(decoded.userId);

  if (!user || !user.isActive) {
    return res.status(401).json({
      success: false,
      message: 'Invalid token or account deactivated.'
    });
  }

  req.user = user;
  next();
};

// Optional Authentication (for public routes that benefit from user
context)
export const optionalAuth = async (req, res, next) => {
  const token = req.header('Authorization')?.replace('Bearer ', '');

  if (token) {
    try {
      const decoded = jwt.verify(token, process.env.JWT_SECRET);
      const user = await User.findById(decoded.userId);
      if (user && user.isActive) {
        req.user = user;
      }
    } catch (error) {
      // Continue without authentication
    }
  }

  next();
};
```

Password Security

```
// Password Hashing (bcrypt with salt rounds: 12)
userSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next();

  const salt = await bcrypt.genSalt(12);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

// Password Comparison
userSchema.methods.comparePassword = async function(candidatePassword) {
  return bcrypt.compare(candidatePassword, this.password);
};
```

Security Headers (Helmet)

```
// Helmet provides these security headers:
- Content-Security-Policy
- X-DNS-Prefetch-Control
- X-Frame-Options
- X-Content-Type-Options
- Strict-Transport-Security
- X-Download-Options
- X-Permitted-Cross-Domain-Policies
- Referrer-Policy
- X-XSS-Protection
```

Rate Limiting

```
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000,  // 15 minutes
  max: 100,                   // 100 requests per window per IP
  message: {
    success: false,
    message: 'Too many requests from this IP, please try again later.'
  }
});
```

Input Validation

```
// Using express-validator
export const validateRegister = [
  body('name')
    .trim()
    .isLength({ min: 2, max: 50 })
    .withMessage('Name must be between 2 and 50 characters'),
```

```
  body('email')
    .isEmail()
    .normalizeEmail()
    .withMessage('Please provide a valid email'),
  body('password')
    .isLength({ min: 6 })
    .withMessage('Password must be at least 6 characters long'),
  body('phone')
    .optional()
    .matches(/^[6-9]\d{9}$/)
    .withMessage('Please provide a valid phone number')
];

// ObjectId Validation
export const validateObjectId = (field) => {
  return (req, res, next) => {
    const value = req.params[field] || req.body[field];

    if (value && !/^[0-9a-fA-F]{24}$/.test(value)) {
      return res.status(400).json({
        success: false,
        message: `Invalid ${field} format`
      });
    }

    next();
  };
};
```

---

9. State Management

Context Architecture

```
┌─────────────────────────────────────────────────────────────────┐
│                            App Layout                            │
│  ┌───────────────────────────────────────────────────────────┐  │
│  │                        AuthProvider                        │  │
│  │  ┌─────────────────────────────────────────────────────┐  │  │
│  │  │                    CartProvider                      │  │  │
│  │  │  ┌───────────────────────────────────────────────┐  │  │  │
│  │  │  │              WishlistProvider                  │  │  │  │
│  │  │  │  ┌─────────────────────────────────────────┐  │  │  │  │
│  │  │  │  │            Page Components              │  │  │  │  │
│  │  │  │  └─────────────────────────────────────────┘  │  │  │  │
│  │  │  └───────────────────────────────────────────────┘  │  │  │
│  │  └─────────────────────────────────────────────────────┘  │  │
│  └───────────────────────────────────────────────────────────┘  │
└─────────────────────────────────────────────────────────────────┘
```

AuthContext

```
interface AuthContextType {
  user: User | null;
  token: string | null;
  isLoading: boolean;
  isAuthenticated: boolean;
  login: (email: string, password: string) => Promise<void>;
  register: (userData: RegisterData) => Promise<void>;
  logout: () => void;
  updateUser: (userData: Partial<User>) => Promise<void>;
}

// Usage
const { user, isAuthenticated, login, logout } = useAuth();

// Features:
- Persists auth state in localStorage
- Auto-validates token on app load
- Clears all user data on logout
- Provides loading state during auth operations
```

CartContext

```
interface CartContextType {
  items: CartItem[];
  total: number;
  itemCount: number;
  isLoading: boolean;
  addToCart: (product: Product, quantity: number, size: string) =>
Promise<void>;
  updateQuantity: (index: number, quantity: number) => Promise<void>;
  removeFromCart: (index: number) => Promise<void>;
  clearCart: () => Promise<void>;
  refreshCart: () => Promise<void>;
}

// Usage
const { items, total, itemCount, addToCart, removeFromCart } = useCart();

// Features:
- Syncs with backend API when authenticated
- Falls back to localStorage when not authenticated
- Auto-calculates totals
- Provides item count for badge display
```

WishlistContext

```
interface WishlistContextType {
  items: string[];              // Product IDs
  isLoading: boolean;
  isInWishlist: (productId: string) => boolean;
  addToWishlist: (productId: string) => void;
  removeFromWishlist: (productId: string) => void;
  toggleWishlist: (productId: string) => void;
}

// Usage
const { items, isInWishlist, toggleWishlist } = useWishlist();

// Features:
- Syncs with user data when authenticated
- Falls back to localStorage when not authenticated
- Provides toggle functionality for UI
```

Local Storage Keys

Key	Description	Format
`authToken`	JWT authentication token	string
`user`	Cached user object	JSON
`cart`	Cart items (guest users)	JSON
`wishlist`	Wishlist product IDs	JSON array
`wardrobe.category`	Last selected wardrobe category	string
`wardrobe.subtype`	Last selected wardrobe subtype	string
`wardrobe.items`	Wardrobe items cache	JSON array

---

10. Performance Optimizations

Custom Performance Hooks

```
// Debounce Hook - for search inputs
export function useDebounce<T>(value: T, delay: number): T {
  const [debouncedValue, setDebouncedValue] = useState<T>(value);

  useEffect(() => {
    const handler = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    return () => clearTimeout(handler);
  }, [value, delay]);
```

```
  return debouncedValue;
}

// Usage
const debouncedSearch = useDebounce(searchQuery, 300);

// Throttle Hook - for scroll/resize events
export function useThrottle<T extends (...args: unknown[]) => unknown>(
  callback: T,
  delay: number
): T;

// Intersection Observer Hook - for lazy loading
export function useIntersectionObserver({
  threshold = 0.1,
  rootMargin = "0px"
} = {}) {
  const [isIntersecting, setIsIntersecting] = useState(false);
  const [hasIntersected, setHasIntersected] = useState(false);
  const ref = useRef<HTMLElement>(null);
  // ...
  return { ref, isIntersecting, hasIntersected };
}

// Virtual Scrolling Hook - for large lists
export function useVirtualScroll({
  itemHeight,
  containerHeight,
  itemCount,
  overscan = 5
}) {
  // Returns visible items with positions
  return { visibleItems, totalHeight, setScrollTop };
}
```

Loading States

```
// Skeleton Components
<ProductSkeleton />     // Product card placeholder
<CardSkeleton />        // Generic card placeholder
<LoadingSkeleton />     // Basic skeleton element

// Loading Spinner
<LoadingSpinner size="sm" />  // h-4 w-4
<LoadingSpinner size="md" />  // h-6 w-6
<LoadingSpinner size="lg" />  // h-8 w-8
```

Image Optimization

```
// Cloudinary Optimization
export const getOptimizedImageUrl = (
  publicId: string,
  options: {
    width?: number;      // Default: 400
    height?: number;     // Default: 400
    quality?: number;    // Default: 80
    format?: string;     // Default: 'auto'
  } = {}
): string => {
  return cloudinary.url(publicId, {
    ...options,
    crop: 'fill',
    gravity: 'auto'
  });
};

// Lazy Image Component
<LazyImage
  src={imageUrl}
  alt="Product"
  placeholder={<LoadingSkeleton className="aspect-square" />}
/>
```

Backend Optimizations

```
// Database Indexes
userSchema.index({ email: 1 }, { unique: true });
userSchema.index({ createdAt: -1 });

productSchema.index({ name: 'text', description: 'text', brand: 'text' });
productSchema.index({ category: 1, subcategory: 1 });
productSchema.index({ price: 1 });
productSchema.index({ rating: -1 });

orderSchema.index({ user: 1, createdAt: -1 });
orderSchema.index({ orderStatus: 1 });

reviewSchema.index({ user: 1, product: 1 }, { unique: true });
reviewSchema.index({ product: 1, createdAt: -1 });

// Pagination
export const getPaginationOptions = (req: Request) => {
  const page = parseInt(req.query.page as string) || 1;
  const limit = Math.min(parseInt(req.query.limit as string) || 10, 100);
  const skip = (page - 1) * limit;
  return { page, limit, skip };
};

// Response Compression
```

```
app.use(compression());

// Lean Queries (for read-only operations)
const products = await Product.find(query).lean();
```

---

11. Deployment Guide

Prerequisites

- Node.js 18+
- MongoDB 6+
- Cloudinary account
- Domain with SSL certificate

Backend Deployment

```bash
# 1. Clone repository
git clone <repository-url>
cd backend

# 2. Install dependencies
npm install

# 3. Set environment variables
cp .env.example .env
# Edit .env with production values

# 4. Build TypeScript
npm run build

# 5. Start production server
npm start
```

Frontend Deployment

```bash
# 1. Navigate to frontend
cd frontend

# 2. Install dependencies
npm install

# 3. Set environment variables
echo "NEXT_PUBLIC_API_URL=https://api.yourdomain.com/api" > .env.local

# 4. Build for production
npm run build
```

```
# 5. Start production server
npm start
```

Docker Deployment

```
# Backend Dockerfile
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY dist ./dist
EXPOSE 5001
CMD ["node", "dist/server.js"]

# Frontend Dockerfile
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:18-alpine AS runner
WORKDIR /app
COPY --from=builder /app/.next ./.next
COPY --from=builder /app/public ./public
COPY --from=builder /app/package.json ./
RUN npm ci --only=production
EXPOSE 3000
CMD ["npm", "start"]
```

Environment Configuration

```
# Production Environment Variables

# Backend
NODE_ENV=production
PORT=5001
MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/fittingly
JWT_SECRET=<strong-random-secret-256-bits>
JWT_EXPIRE=7d
CLOUDINARY_CLOUD_NAME=<your-cloud-name>
CLOUDINARY_API_KEY=<your-api-key>
CLOUDINARY_API_SECRET=<your-api-secret>
FRONTEND_URL=https://yourdomain.com
RATE_LIMIT_WINDOW_MS=900000
RATE_LIMIT_MAX_REQUESTS=100
```

```
# Frontend
NEXT_PUBLIC_API_URL=https://api.yourdomain.com/api
```

Health Checks

```
# Backend Health Check
curl https://api.yourdomain.com/health

# Response
{
  "success": true,
  "message": "Server is running",
  "timestamp": "2025-01-15T10:30:00.000Z",
  "environment": "production"
}

# API Info
curl https://api.yourdomain.com/api

# Response
{
  "success": true,
  "message": "Fittingly API is running",
  "version": "1.0.0",
  "endpoints": {
    "auth": "/api/auth",
    "products": "/api/products",
    "wardrobe": "/api/wardrobe",
    "cart": "/api/cart",
    "orders": "/api/orders",
    "reviews": "/api/reviews",
    "upload": "/api/upload"
  }
}
```

Appendix A: Error Codes

HTTP Code	Error Type	Description
400	Bad Request	Invalid input data or validation error
401	Unauthorized	Missing or invalid authentication token
403	Forbidden	Insufficient permissions
404	Not Found	Resource not found
409	Conflict	Duplicate resource (e.g., email already exists)
429	Too Many Requests	Rate limit exceeded

HTTP Code	Error Type	Description
500	Internal Server Error	Server-side error

Appendix B: Product Categories

Main Categories

- `men` - Men's fashion
- `women` - Women's fashion
- `kids` - Kids' fashion
- `unisex` - Unisex items

Subcategories

- `topwear` - T-shirts, shirts, jackets, hoodies
- `bottomwear` - Jeans, trousers, shorts
- `footwear` - Sneakers, formal shoes, loafers
- `accessories` - Watches, sunglasses, bags, belts

Wardrobe Categories

- `upper` - Upper body clothing (T-shirts, shirts, vests, caps)
- `lower` - Lower body clothing (Jeans, trousers, shorts)
- `shoes` - Footwear (Sneakers, formals, loafers)

Appendix C: Payment Methods

Method	Code	Description
Credit Card	`credit_card`	Visa, Mastercard, etc.
Debit Card	`debit_card`	Bank debit cards
UPI	`upi`	Unified Payments Interface
Net Banking	`net_banking`	Online bank transfer
Wallet	`wallet`	Digital wallets
Cash on Delivery	`cod`	Pay on delivery

Appendix D: Order Status Flow

```
pending → confirmed → shipped → delivered
    ↓           ↓           ↓
cancelled  cancelled  (cannot cancel)
```

Status	Description
`pending`	Order placed, awaiting confirmation
`confirmed`	Order confirmed, preparing for shipment
`shipped`	Order shipped, in transit
`delivered`	Order delivered to customer
`cancelled`	Order cancelled (only before shipping)

Appendix E: File Size Limits

Type	Limit
Request Body	10 MB
Single Image Upload	10 MB
Multiple Images	10 MB per file, max 10 files

Documentation generated for Fittingly v1.0.0 Last updated: January 15, 2026